

Dependency Preservation in 3NF vs BCNF

A complete step-by-step conceptual guide with a detailed counterexample

★ **Key Statement:** Every lossless decomposition into 3NF can be made dependency-preserving, but BCNF does not always guarantee dependency preservation.

1. What Happens During Decomposition?

When normalizing a relation to reduce redundancy and anomalies, we decompose (split) one large relation into smaller relations. A **good decomposition** should ideally satisfy two key properties:

1. Lossless Decomposition

Ensures that after decomposing relation R into R_1 and R_2 , joining them back ($R_1 \bowtie R_2$) yields exactly the original relation without losing data or generating spurious tuples.

2. Dependency Preservation

Ensures that all original Functional Dependencies (FDs) can be enforced directly within individual decomposed relations without requiring costly join operations across tables.

2. Which Normal Forms Guarantee What?

Normal Form	Lossless Decomposition	Dependency Preservation
1NF	✓ Generally possible	✓ Generally possible
2NF	✓ Generally possible	✓ Generally possible
3NF	✓ Guaranteed (via Synthesis)	✓ Guaranteed
BCNF	✓ Guaranteed	✗ Not Always Guaranteed

Core Takeaway: 3NF can always achieve a decomposition that is both lossless and dependency-preserving. However, BCNF may force a trade-off where achieving BCNF comes at the cost of losing functional dependencies.

3. Proof by Counterexample: Relation in 3NF but Not BCNF

Consider the relation $R(A, B, C, D)$ with the following set of Functional Dependencies (FDs):

- $AB \rightarrow CD$
- $D \rightarrow A$

Step 1: Finding Candidate Keys

To identify Candidate Keys, examine the Right-Hand Side (RHS) of all FDs:

- RHS attributes present: **A, C, D**
- Attribute missing from RHS: **B** (Must appear in every candidate key)

Testing closures containing attribute **B**:

- **Test (AB)***: $AB \rightarrow CD \Rightarrow (AB)^+ = \{A, B, C, D\}$. Thus, **AB** is a candidate key.
- **Test (BD)***: $D \rightarrow A$ gives $\{A, B, D\}$; then $AB \rightarrow CD$ gives $\{A, B, C, D\}$. Thus, **BD** is also a candidate key.

Candidate Keys: AB, BD

Step 2: Attribute Classification (Prime vs Non-Prime)

Attributes belonging to at least one candidate key are **Prime**; others are **Non-Prime**.

Attribute	Classification	Reasoning
A	Prime	Part of Candidate Key AB
B	Prime	Part of Candidate Keys AB and BD
C	Non-Prime	Does not appear in any candidate key
D	Prime	Part of Candidate Key BD

Step 3: Checking 3NF and BCNF Compliance

3NF Requirement: For every non-trivial FD $X \rightarrow Y$, either X is a Superkey OR Y is a Prime attribute.

- $AB \rightarrow CD$: **AB** is a Superkey. ✓ **Satisfies 3NF**
- $D \rightarrow A$: **D** is not a Superkey, but **A** is a Prime attribute. ✓ **Satisfies 3NF**

→ **Relation R is in 3NF.**

BCNF Requirement: For every non-trivial FD $X \rightarrow Y$, X MUST be a Superkey.

- $D \rightarrow A$: **D** is NOT a Superkey. ✗ **Violates BCNF**

→ **Relation R is NOT in BCNF.**

4. Decomposing Relation R into BCNF

To convert R into BCNF, we decompose using the violating dependency $D \rightarrow A$:

$R_1 (D, A)$
FD: $D \rightarrow A$

$R_2 (B, C, D)$
FD: $BD \rightarrow C$

A. Checking for Lossless Property

- Common attribute between R_1 and R_2 is **D**.
- In R_1 , $D \rightarrow A$ holds, which means **D** is a key for R_1 .

- Since the common attribute forms a key for one of the decomposed relations, the decomposition is ✓ **Lossless**.

B. Checking Dependency Preservation

Let's check if original dependencies ($AB \rightarrow CD$ and $D \rightarrow A$) can be preserved:

1. $D \rightarrow A$: Fully contained in $R_1(D, A)$. ✓ **Preserved**
2. $AB \rightarrow CD$: Attribute **A** is in R_1 while **B, C, D** are in R_2 . Cannot be directly placed in either relation.

Testing if $AB \rightarrow CD$ can be derived from decomposed relations:

The available functional dependencies after decomposition are $D \rightarrow A$ (in R_1) and $BD \rightarrow C$ (derived in R_2).

Now, compute the attribute closure of **AB** using ONLY these enforced dependencies:

```
Start: (AB)+ = {A, B}
• Try applying D → A: Cannot apply (D is missing).
• Try applying BD → C: Cannot apply (D is missing).
Result: (AB)+ = {A, B}
```

Since **C** and **D** are not in $(AB)^+$, the dependency $AB \rightarrow CD$ **CANNOT** be derived!

✗ Result: Dependency Preservation Failed

The BCNF decomposition guarantees Lossless Join, but **FAILS Dependency Preservation**. Enforcing $AB \rightarrow CD$ would now require joining R_1 and R_2 on every INSERT/UPDATE operation.

5. Why 3NF is Often Preferred in Real-World Design

Database design involves balancing redundancy against performance and integrity enforcement:

BCNF Properties

- Stricter; completely eliminates redundancy based on FDs.
- May lose functional dependencies.
- Enforcing lost FDs requires expensive table **JOINS** across updates.

3NF Properties

- Allows minor redundancy (for prime attributes).
- **ALWAYS** guarantees Lossless + Dependency Preservation.
- FDs can be checked locally within a single table with zero JOIN overhead.

⚠ Exam Trap: Avoid Precise Wording Mistakes

Incorrect: "BCNF is not dependency preserving."

Correct: "BCNF does not always guarantee dependency preservation." (Some relations decomposed into BCNF *can* preserve dependencies, but it is not guaranteed for every relation).

One-Line Exam Answer & Memory Trick

Exam Answer: Every relation has a lossless decomposition into BCNF, but BCNF decomposition does not necessarily preserve all functional dependencies. In contrast, 3NF synthesis guarantees both a lossless and dependency-preserving decomposition.

Memory Trick: *"BCNF is stronger for redundancy, but 3NF is safer for dependencies."*